

Chaining (Separate Chaining)

1. Concept

Definition

Chaining is a collision resolution technique where each hash table index stores a linked list of elements that hash to the same value.

Instead of storing only one element per index, we store a **list of elements**.

2. Example

Let:

$$m = 5$$

Hash function:

$$h(k) = k \mod 5$$

Keys:

$$12, 7, 22, 17, 5$$

Compute hash values:

$$h(12) = 2$$

$$h(7) = 2$$

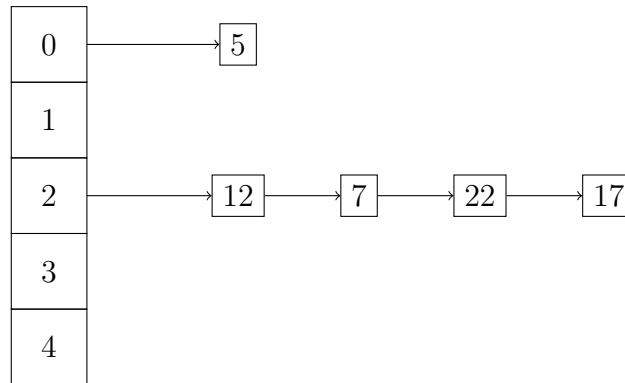
$$h(22) = 2$$

$$h(17) = 2$$

$$h(5) = 0$$

Many keys map to index 2 $\rightarrow$ Collision handled using linked list.

3. Visual Representation



Index 2 stores a linked list of 4 elements.

4. Time Complexity

Let:

- n = number of elements
- m = table size

$$\alpha = \frac{n}{m}$$

where α is called the **Load Factor**.

Insertion

Insert at head of list:

$$O(1)$$

Search / Delete

Worst case (all keys in one bucket):

$$O(n)$$

Average case:

$$O(1 + \alpha)$$

Why?

Because average chain length is:

$$\alpha = \frac{n}{m}$$

5. Important Observations

Key Insight

If α is small (for example $\alpha < 1$), then chaining gives nearly constant time performance.

- Works even if $\alpha > 1$
 - No clustering problem
 - Easy to implement
 - Requires extra memory for linked lists
-

6. Worst-Case Scenario in Chaining

Worst Case: All Keys in One Bucket

If all keys hash to the same index, the hash table degenerates into a linked list.

Suppose:

$$h(k_1) = h(k_2) = \dots = h(k_n)$$

Then the bucket contains:

n elements

Time complexity becomes:

$$\text{Search} = O(n)$$

$$\text{Insert} = O(1) \quad (\text{if inserted at head})$$

$$\text{Delete} = O(n)$$

This is equivalent to having no hashing at all.

Visual Intuition:

Instead of:

m small linked lists

We get:

1 long linked list

Which destroys the benefit of hashing.

Dynamic Resizing (Rehashing)

Why Resize?

As the number of elements increases, load factor increases, leading to longer chains and slower searches.

Load factor:

$$\alpha = \frac{n}{m}$$

When:

$$\alpha > 1 \quad (\text{or chosen threshold})$$

Performance degrades.

Solution: Rehashing

Steps:

1. Create a new larger table (usually $2\times$ size)
2. Choose a new prime table size
3. Recompute hash values for all elements
4. Insert them into new table

Time Complexity of Rehashing:

$$O(n)$$

But since resizing happens occasionally, the amortized cost per insertion remains:

$$O(1)$$

Practical Thresholds:

- Resize when $\alpha > 1$ (common in chaining)
- Some implementations resize at $\alpha > 0.75$

Chaining Works Well — If Load Factor Is Controlled

Chaining = Flexible and Robust Collision Handling